

COGNIZANT DIGITAL NURTURE 5.0

PL/SQL EXERCISE 1 – CONTROL STRUCTURES

SCENARIO 1: APPLY DISCOUNT TO LOAN INTEREST RATES

Problem Statement:

The bank wants to apply a 1% discount to loan interest rates for customers above 60 years old.

PL/SQL Code:

```
SET SERVEROUTPUT ON;

BEGIN
FOR cust IN (
SELECT CustomerID, CustomerName, Age
FROM Customers
) LOOP
    IF cust.Age > 60 THEN
        UPDATE Loans
        SET InterestRate = InterestRate - 1
        WHERE CustomerID = cust.CustomerID;

        DBMS_OUTPUT.PUT_LINE(
            'Discount applied for customer: ' || cust.CustomerName
        );
    END IF;
END LOOP;

COMMIT;
END;
/
```

Output Screenshot:

The screenshot shows the SQL Developer interface. The top toolbar includes icons for running queries, saving, and other standard database operations. The main window is divided into two panes: 'Worksheet' and 'Query Builder'. The 'Query Builder' pane contains the following PL/SQL code:

```

SET SERVEROUTPUT ON;
BEGIN
  FOR cust IN (
    SELECT CustomerID, CustomerName, Age
    FROM Customers
  ) LOOP
    IF cust.Age > 60 THEN
      UPDATE Loans
      SET InterestRate = InterestRate - 1
      WHERE CustomerID = cust.CustomerID;
      DBMS_OUTPUT.PUT_LINE(
        'Discount applied for customer: ' || cust.CustomerName
      );
    END IF;
  END LOOP;
  COMMIT;
END;
/

SELECT * FROM Loans;

```

The 'Query Result' pane at the bottom shows the output of the query. It displays a table with 4 columns: LOANID, CUSTOMERID, INTERESTRATE, and DUE DATE. The table contains 3 rows of data:

LOANID	CUSTOMERID	INTERESTRATE	DUE DATE
1	501	101	6 06-07-26
2	502	102	12 05-08-26
3	503	103	7 11-07-26

EXECUTION OUTPUT:

The screenshot shows the 'Script Output' window in SQL Developer. It displays the following text:

```

PL/SQL procedure successfully completed.

Discount applied for customer: Ravi
Discount applied for customer: Kumar

PL/SQL procedure successfully completed.

```

SCENARIO 2: PROMOTE CUSTOMERS TO VIP STATUS

Problem Statement:

A customer can be promoted to VIP status based on their balance.

PL/SQL Code:

```
SET SERVEROUTPUT ON;

BEGIN
FOR cust IN (
SELECT CustomerID, CustomerName, Balance
FROM Customers
) LOOP

    IF cust.Balance > 10000 THEN

        UPDATE Customers

        SET IsVIP = 'TRUE'

        WHERE CustomerID = cust.CustomerID;

        DBMS_OUTPUT.PUT_LINE(
            'VIP Status Granted to: ' || cust.CustomerName
        );

    END IF;

END LOOP;

COMMIT;
END;
/
```

Output Screenshot:

The screenshot shows the SQL Developer interface. The top pane is the 'Query Builder' with a PL/SQL procedure script. The script sets server output on, loops through customers, and updates the 'IsVIP' status for those with a balance greater than 10,000. The bottom pane shows the 'Script Output' window with the message 'PL/SQL procedure successfully completed.' and the output of the DBMS_OUTPUT.PUT_LINE statements: 'VIP Status Granted to: Ravi' and 'VIP Status Granted to: Kumar'.

```
SET SERVEROUTPUT ON;
BEGIN
  FOR cust IN (
    SELECT CustomerID, CustomerName, Balance
    FROM Customers
  ) LOOP
    IF cust.Balance > 10000 THEN
      UPDATE Customers
      SET IsVIP = 'TRUE'
      WHERE CustomerID = cust.CustomerID;
      DBMS_OUTPUT.PUT_LINE(
        'VIP Status Granted to: ' || cust.CustomerName
      );
    END IF;
  END LOOP;
  COMMIT;
END;
```

Task completed in 0.055 seconds

PL/SQL procedure successfully completed.

VIP Status Granted to: Ravi
VIP Status Granted to: Kumar

PL/SQL procedure successfully completed.

FINAL OUTPUT

The screenshot shows the SQL Developer interface with the 'Query Result' window. The query 'SELECT * FROM Customers;' has been executed, and the results are displayed in a table. The table has five columns: CUSTOMERID, CUSTOMERNAME, AGE, BALANCE, and ISVIP. There are three rows of data.

	CUSTOMERID	CUSTOMERNAME	AGE	BALANCE	ISVIP
1	101	Ravi	65	15000	TRUE
2	102	Priya	45	8000	FALSE
3	103	Kumar	70	20000	TRUE

SCENARIO 3: LOAN DUE REMINDERS

Problem Statement:

The bank wants to send reminders to customers whose loans are due within the next 30 days.

PL/SQL Code:

```
SET SERVEROUTPUT ON;
```

```
BEGIN
```

```
FOR loan_rec IN (
```

```
SELECT c.CustomerName,
```

```
l.LoanID,
```

```
l.DueDate
```

```
FROM Customers c
```

```
JOIN Loans l
```

```
ON c.CustomerID = l.CustomerID
```

```
WHERE l.DueDate <= SYSDATE + 30
```

```
) LOOP
```

```
    DBMS_OUTPUT.PUT_LINE(
```

```
        'Reminder: Loan ' || loan_rec.LoanID ||
```

```
        ' for customer ' || loan_rec.CustomerName ||
```

```
        ' is due on ' || TO_CHAR(loan_rec.DueDate,'DD-MON-YYYY')
```

```
    );
```

```
END LOOP;
```

```
END;
```

```
/
```

Output Screenshot:

The screenshot displays the SQL Developer interface. The top pane shows a PL/SQL procedure in the Query Builder. The procedure sets server output on, begins a loop, and iterates through loans due within 30 days. It sends reminders for Loan 501 (Ravi) and Loan 503 (Kumar). The bottom pane shows the script output, confirming successful completion and displaying the reminder messages.

```
SET SERVEROUTPUT ON;
BEGIN
  FOR loan_rec IN (
    SELECT c.CustomerName,
           l.LoanID,
           l.DueDate
    FROM Customers c
    JOIN Loans l
    ON c.CustomerID = l.CustomerID
    WHERE l.DueDate <= SYSDATE + 30
  ) LOOP
    DBMS_OUTPUT.PUT_LINE(
      'Reminder: Loan ' || loan_rec.LoanID ||
      ' for customer ' || loan_rec.CustomerName ||
      ' is due on ' || TO_CHAR(loan_rec.DueDate, 'DD-MON-YYYY')
    );
  END LOOP;
END;
```

Script Output x | Query Result x | Query Result 1 x

Task completed in 0.084 seconds

PL/SQL procedure successfully completed.

Reminder: Loan 501 for customer Ravi is due on 06-JUL-2026
Reminder: Loan 503 for customer Kumar is due on 11-JUL-2026

PL/SQL procedure successfully completed.

FINALOUTPUT:

The screenshot displays the SQL Developer interface. The top pane shows a query in the Query Builder. The bottom pane shows the query result, which is a table with 4 columns: LOANID, CUSTOMERID, INTERESTRATE, and DUE DATE. The table contains 2 rows of data.

```
SELECT *
FROM Loans
WHERE DueDate <= SYSDATE + 30;
```

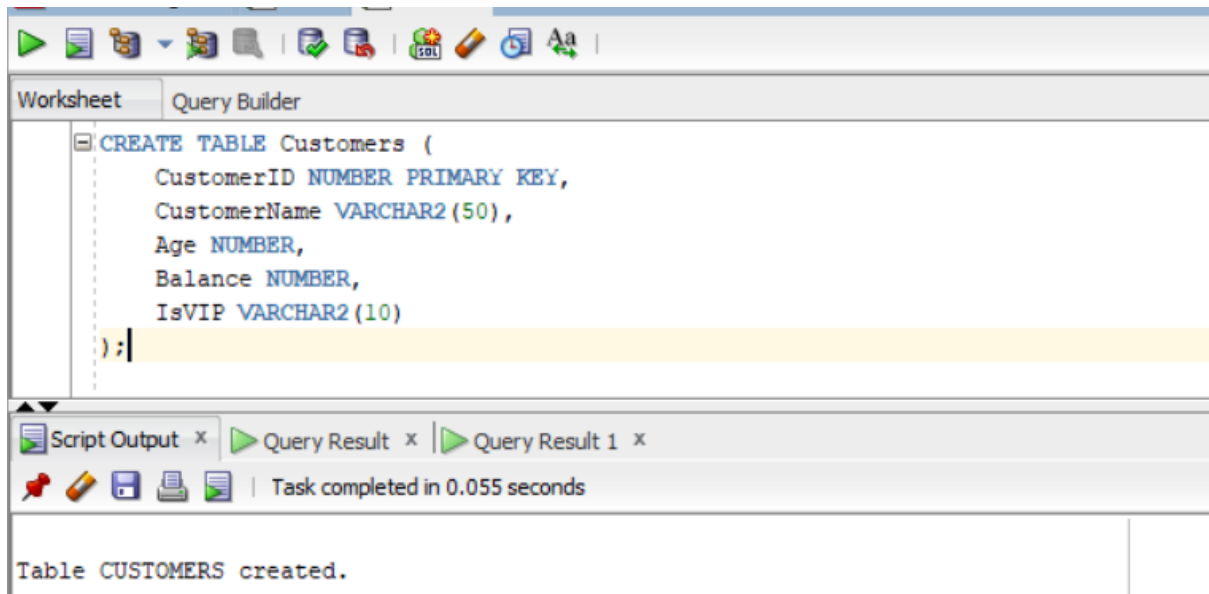
Script Output x | Query Result x | Query Result 1 x

All Rows Fetched: 2 in 0.008 seconds

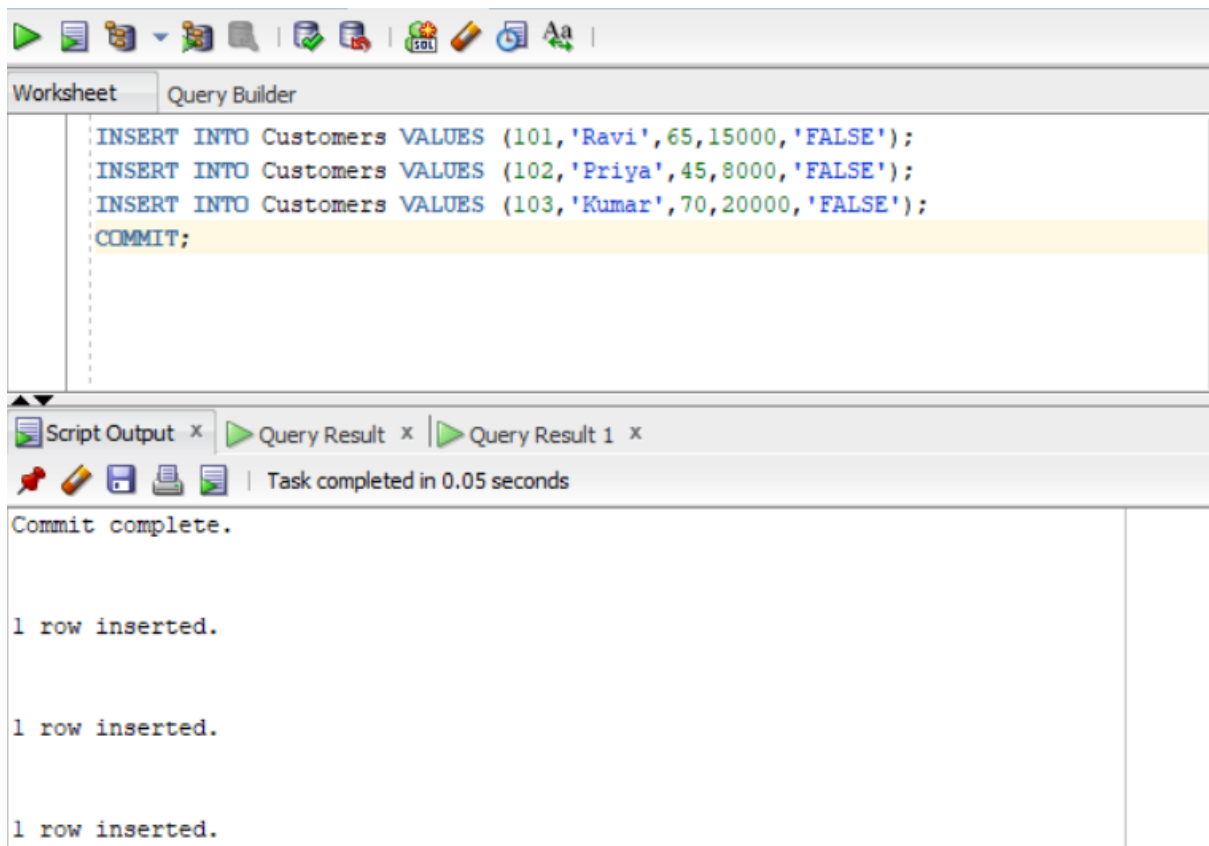
	LOANID	CUSTOMERID	INTERESTRATE	DUE DATE
1	501	101	6	06-07-26
2	503	103	7	11-07-26

DATABASE_SETUP:

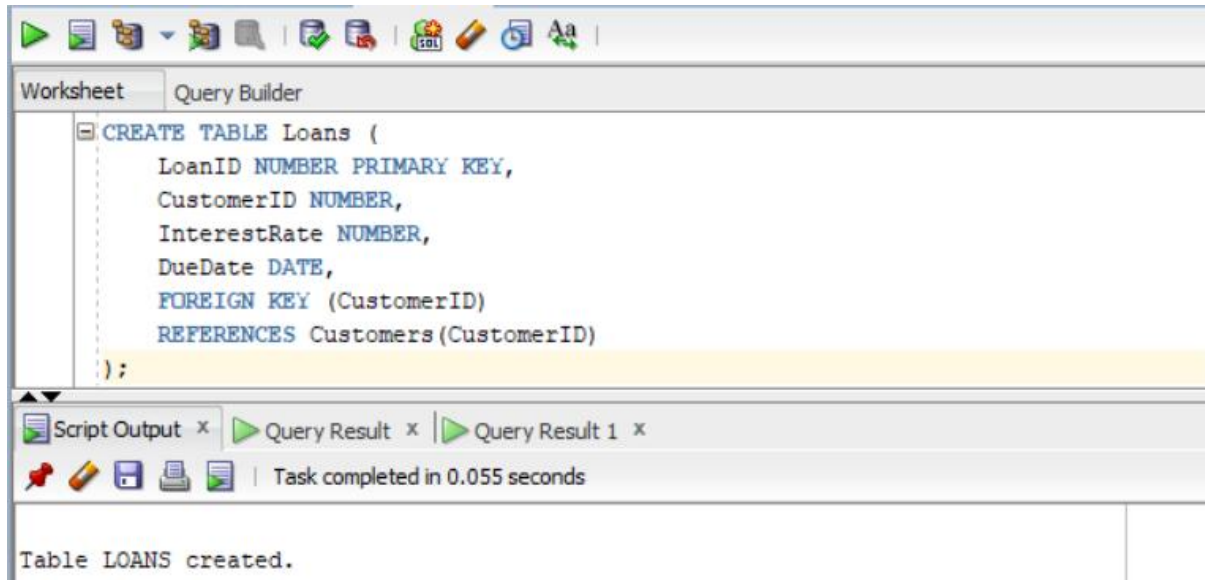
CUSTOMER TABLE CREATION:



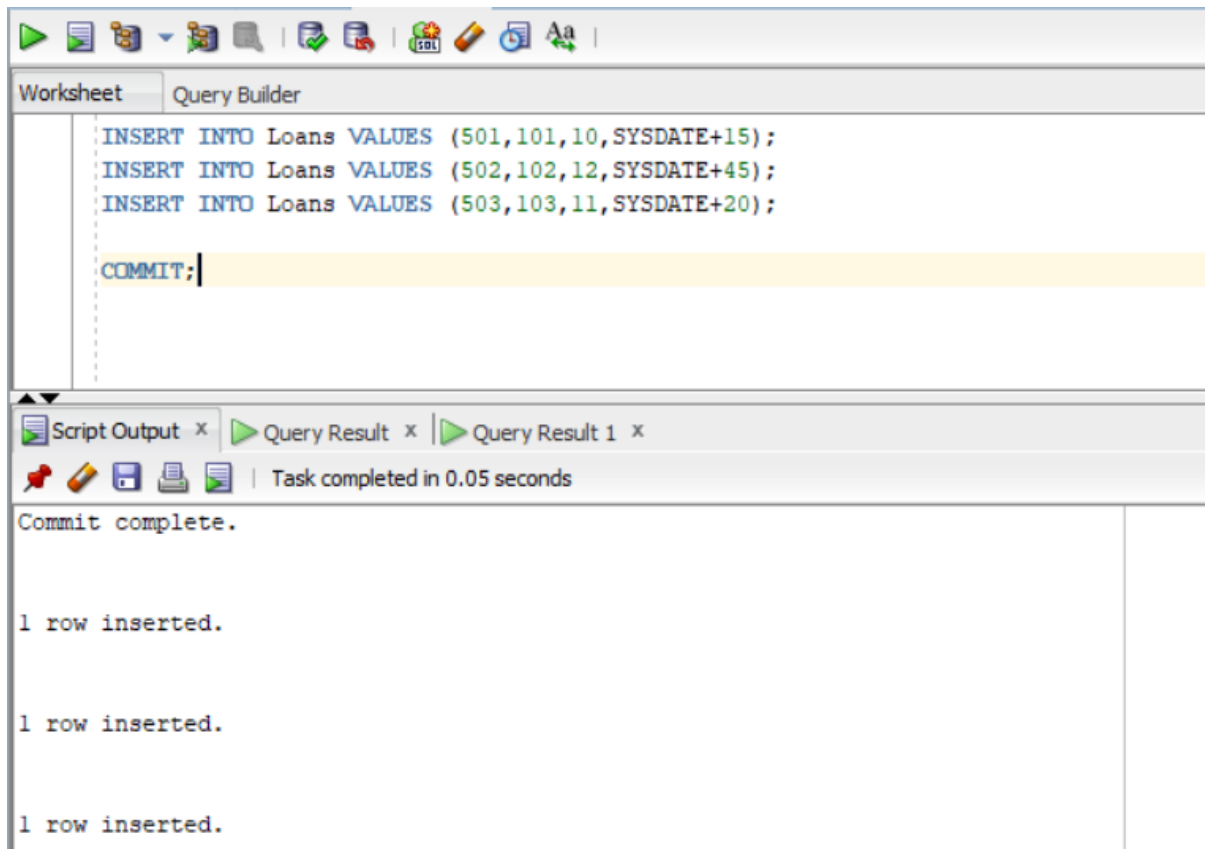
INSERTING ROWS:



LOAN TABLE CREATION:



INSERTING ROWS:



RESULT:

Thus, the PL/SQL programs using control structures were successfully implemented and executed.